



МИНИСТЕРСТВО ОБРАЗОВАНИЯ МОСКОВСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение Московской области «Люберецкий техникум имени Героя Советского Союза, летчика-космонавта Ю.А.Гагарина»

Курсовой проект

по теме: сайт по разработке дизайна персонажа

МДК 08.01 МДК.08.01 Проектирование и разработка интерфейсов пользователя

МДК.09.01 Проектирование и разработка веб-приложений

Специальность: 09.02.07 «Информационные системы и программирование»

Группа № ИС-24, курс 2

Выполнил _____ Я.В. Насыпаева

Руководитель _____ Ю.В. Жирнова

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Актуальность исследования.....	3
Объект исследования.....	3
Цель и задачи работы.....	3
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ. Документооборот в организации.....	3
1.1 Описание предметной области.....	4
1.2 Обзор существующих решений (аналоги).....	5
1.3 Формирование требований к системе.....	5
2 ПРОЕКТНАЯ ЧАСТЬ. Реализация системы документооборота на Django.....	6
2.1 Проектирование архитектуры приложения (компонентная диаграмма UML).....	6
2.3 Реализация функциональных модулей (основные листинги).....	8
2.4 Обеспечение безопасности и авторизации.....	9
2.5 Тестирование и отладка.....	10
3 АНАЛИТИЧЕСКАЯ ЧАСТЬ. САЙТ ДЛЯ ИГРОВОГО СООБЩЕСТВА.....	11
3.1 Описание предметной области.....	11
3.2 Обзор существующих решений.....	11
3.3 Формирование требований к системе.....	12
4 ПРОЕКТНАЯ ЧАСТЬ. Реализация индивидуальной темы на Laravel.....	14
4.1 Проектирование архитектуры приложения (компонентная диаграмма).....	14
4.2 Проектирование базы данных (ER-диаграмма).....	15
4.3 Реализация функциональных модулей (листинги).....	15
4.4 Реализация процедур и триггеров.....	17
4.5 Обеспечение безопасности и авторизации.....	17
4.6 Тестирование и отладка.....	17
ЗАКЛЮЧЕНИЕ.....	19
Результат работы.....	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	21

ВВЕДЕНИЕ

Актуальность исследования

Особый интерес в профессиональном сообществе вызывают два мощных инструмента: Django (основанный на языке Python) и Laravel (основанный на PHP). Оба фреймворка занимают лидирующие позиции в своих экосистемах, предлагают концепцию «батарейки в комплекте» (batteries-included), но при этом имеют принципиально разные архитектурные паттерны и подходы к решению одних и тех же задач. Актуальность определяется теоретической и практической ценностью для обучения в направлении веб-разработки и познании различных способов разработки.

Объект исследования

Объектом исследования является процесс разработки веб-приложений с использованием серверных фреймворков, предметом исследования — функциональные, архитектурные и эксплуатационные особенности фреймворков Django и Laravel.

Цель и задачи работы

Цель данной курсовой работы заключается в освоении двух фреймворков (Django и Laravel).

Для достижения поставленной цели необходимо решить следующие задачи:

1. Рассмотреть теоретическую и практическую базы Django и Laravel.
2. Выполнить развертывание рабочих сред для обоих фреймворков и описать процесс создания базового каркаса приложения.
3. Провести сравнительный анализ ключевых аспектов работы с базами данных, маршрутизацией, аутентификацией пользователей и middleware.
4. Добавить развернутый функционал на сайт, прописать процедуры и триггеры.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ. Документооборот в организации

1.1 Описание предметной области

frontend	backend
визуальная часть сайта HTML-CSS-JS КОНТЕНТ(текст, медиа, интерактивные элементы, стили, анимации)	функциональная часть сайта PHP-PY-JS (C#, C++, GO) ФУНКЦИИ(обработка запросов, работа с файлами, API)

Таблица 1 - Структура сайта

Функции сайта:

1. Подключение к БД;
2. Авторизация;
3. Работа с данными (просмотр, (поиск, сортировка, фильтрация), добавление, редактирование, удаление);
4. Админпанель;
5. Динамическое изменение HTML-страниц;
6. Асинхронность (когда к одной базе одновременно подключается несколько пользователей и выполняют разную работу с данными);
7. Документация.

Веб-сервер – это ПК или ПО, которое принимает запросы от браузера и отправляет результат в виде файлов пользователю.

Чтобы обработать запрос (см. таб. 2), нужно прописать правила: какой формат запроса и ответа. Эти правила назвали протоколом (HTTP, HTTPS, QIC)[1].

ТИПЫ ЗАПРОСОВ		
GET	POST	остальные
получать вывод данных	публиковать заполнение форм	put, patch, delete редактировать

Таблица 2 - Типы запросов

1.2 Обзор существующих решений (аналоги)

Framework – готовый набор инструментов, который реализует описанные выше функции

PHP	PY	JS
LARAVEL YII	DJANGO FAST API FLASK	REACT NOD

Таблица 3 - Фреймворки

DJANGO – фреймворк на языке питон для создания бекенд части сайта.

1.3 Формирование требований к системе

В ред ос язык питон встроен, однако для установки дополнительных инструментов нужно поставить менеджер пакетов `pip` [2]. `pip install django` - скачивает фреймворк из интернета и устанавливает на компьютер. Эта команда создает папки для запуска сайта - `django-admin startproject siteNYV`. `python manage.py runserver` - Запускает сайт. Сайт запускается локально на компьютере, но чтобы увидеть его нужно перейти по ссылке, которая пишется в терминале после запуска локалхоста: <http://127.0.0.1:8000/>. Цифры показывают что джанго запускается на том же компьютере, что и код, 8000 – это порт, он нужен чтобы на веб-сервере можно было запускать несколько программ.

паттерны проектирование		
MVT	ДЖАНГО	model - данные view - логика взаимодействия template - внешний вид сайта
MVC	ЛАРАВЕЛ	modal - данные view - внешний вид сайта controller - логика взаимодействия
MVVM	АНДРОИД	modal - данные view - внешний вид сайта ViewModel - логика взаимодействия
название	фреймворки	расшифровка

Таблица 4 - Паттерны проектирования

2 ПРОЕКТНАЯ ЧАСТЬ. Реализация системы документооборота на Django

2.1 Проектирование архитектуры приложения (компонентная диаграмма UML)

UML (Unified Modeling Language) - унифицированный язык моделирования.

Структура use case диаграмм (см. рис. 1).

1. Овалы – прецеденты (use case), функции системы, действия, которые может выполнять пользователь. Текст в овалах приводит к единому стилю (глаголы или отглагольные существительные, вход в систему или войти в систему), принято начинать с роли, у которой мало действий и идти на увеличение.
2. Прямоугольники – система, объединяет овалы в приложения (веб-сайт, тг бот). в прямоугольнике указывается название системы.
3. Стрелки – связи. На диаграмме должно быть три вида стрелки: обычные, пунктирные (includes) включать - обязательное действие, пунктирные (extends) расширять - доп. действие.
4. Человечки – роль пользователя (actor). Обычно в системе выделяют пользователя и администратора. В зависимости от тематики сайта пользователя называют клиентом и тд. Администратором может быть преподаватель, менеджер и тд. дополнительно можно добавить неавторизованного пользователя (гость), если на сайте есть функции доступные до авторизации (просмотр данных).
5. Цилиндр – бд. Все надписи должны быть на русском. Как только на сайте есть авторизация в БД добавляются таблицы “пользователи” и ”роли”. В них прописывается какие данные нужны на сайте (фио, телефон, почта).

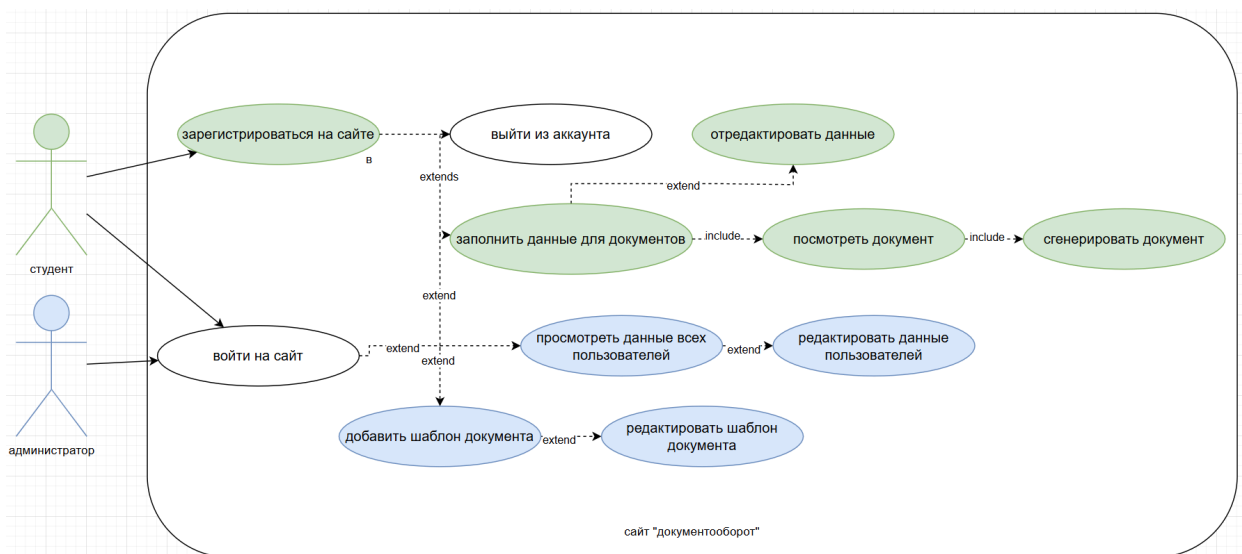


рисунок 1 - Основные функции

2.2 Проектирование базы данных (ER-диаграмма, описание таблиц)

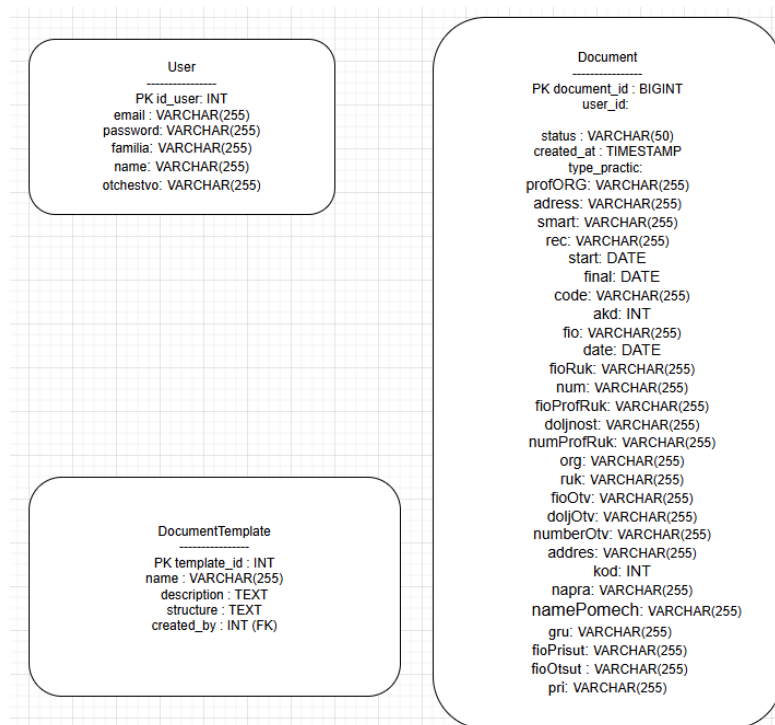


рисунок 2 - ER диаграмма

ER-модель (см. рис. 2) — модель данных, позволяющая описывать концептуальные схемы предметной области [3]. Таблица User содержит в себе информацию о пользователе, который регистрируется на сайте, таблица Document содержит в себе все переменные из документов, для внесения данных в последствие в сам документ для печати. DocumentTemplate - таблица, содержащая в себе данные о шаблоне документа.

2.3 Реализация функциональных модулей (основные листинги)

В джанго при создании проекта автоматически создается файл БД. Подключение к нему прописано в файле `sattings.py`. по умолчанию используется локальная база `sqlite`, в дальнейшем ее можно поменять на любую удаленную. По умолчанию БД пустая, однако, `python manage.py migrate` создает таблицы. Модели в Django — это основной источник данных приложения [4]. Она выступает в роли связующего звена между кодом на Python и базой данных, выполняя роль чертежа для создания таблиц

и управления ими. Для создания модели используется следующая команда:
`python manage.py inspectdb document >> mainNYV/models.py.`

Для работы с базой в джанго создана админ панель. Чтобы в нее войти создаем админа командой `createsuperuser` и переходим по ссылке <http://127.0.0.1:8000/admin>.

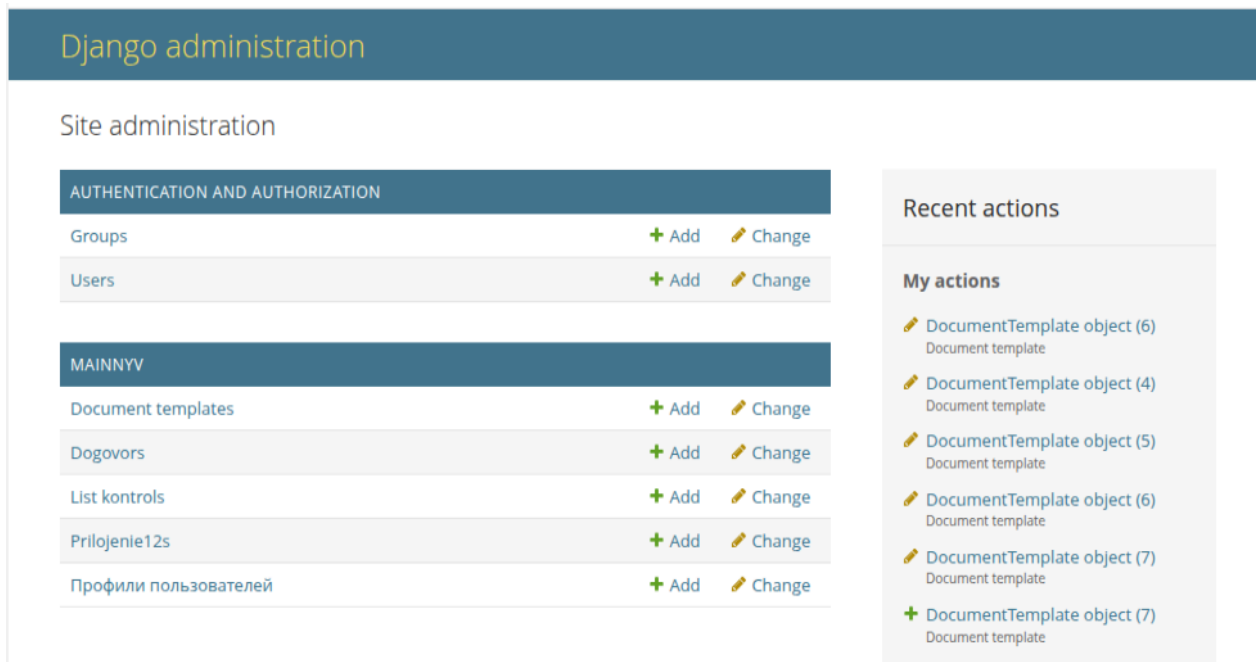


Рисунок 3 - админпанель

Листинг 1 - подключение базы данных к сайту:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

2.4 Обеспечение безопасности и авторизации

register.html

Для работы регистрации нужно, чтобы пользователь мог вносить данные в форму `{{ form }}` [5, 6].

Для отправки данных в БД нужна кнопка `<button>зарегистрироваться</button>` и метод `post`. В джанго встроена

система безопасности и чтобы корректно передавались пароли используются csrf токен `{% csrf_token %}`.

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation:
Enter the same password as before, for verification.

зарегистрироваться

Рисунок 4 - форма регистрации

Листинг 2 - пример регистрации:

```
<form method="POST">
    {% csrf_token %}
    {{form}}
    <button type="submit">зарегистрироваться</button>
</form>
```

views.py

В функцию `loginf` добавили проверку на метод `post`, пока пользователь не введет данные, занесенные в БД, выводить страницу `login`, джанго позволяет не использовать `sql` запрос. В

джанго таблицы сохраняются как классы а столбцы как переменные.

Username:

Password:

войти

Рисунок 5 - форма входа

Листинг 3 - функция обработки входа

```
def loginf(request):
    form = AuthenticationForm(request, data=request.POST or
None)
    if request.method == 'POST' and form.is_valid():
        login(request, form.get_user())
        return redirect('profile')
return render(request, 'login.html', {'form': form})
```

2.5 Тестирование и отладка

`echo "# django_NYV" >> README.md` - создание readme файла, в котором название репозитория. Создает папку `.git`, в которой хранятся системные файлы репозитория - `git add .` Для добавления всех файлов ставится точка, если нужен конкретный файл пишется его имя. В статусе (`git status`) добавленные файлы должны гореть зеленым. `git commit -m "name of commit"`

Коммит – это фиксация, сохранение состояния проекта с каким то именем. Между этими сохранениями можно переключаться. Текст в кавычках – имя изменения.

`git branch -M main` - указываем имя для главной ветки. Ветки нужны если несколько разработчиков параллельно работают над продуктом. Пример: в сайте джанго 1 делает документ дневник, а второй – отчет.

`git remote add origin https://github.com/metyloran/django\_NYV.git` - подключает репозиторий.

`git push https://ghp\_nq4lA7woYrOx3qMb7QcE71MtSSiNSS33DOsV@github.com/metyloran/django\_NYV.git` - Отправляет данные гитхаб. Команда `git pull origin [ветка]` скачивает обновления из удаленного репозитория (например, GitHub или GitLab) и сразу объединяет их с вашей текущей локальной веткой. Это позволяет синхронизировать ваш код с работой других разработчиков.

В итоговом результате сайт принимает от пользователя данные, после чего вносит их в документ, выдавая его для скачивания и последующей распечатки.

<u>ГБПОУ МО «Люберецкий техникум имени Героя Советского Союза, лётчика-космонавта Ю.А. Гагарина»</u>	<u>Цифровое равенство</u>
Адрес: 140002, Московская область, г. Люберцы, Октябрьский проспект, д.114. Тел./факс: (495) 503-45-77 E-mail: <u>mo_gagarintech@mosreg.ru</u> ИНН 5027037310 КПП 502701001 ОГРН 1035005025714 ОКТМО 46748000 д/с20014844300 Министерство экономики и финансов Московской области (ГБПОУ МО "Люберецкий техникум имени Героя Советского Союза, лётчика-космонавта Ю.А. Гагарина") р/с 03224643460000004800 ГУ Банка России по ЦФО//УФК по Московской области г.Москва БИК 004525987 Кор.счет 40102810845370000004	Адрес: <u>г. Томилино</u> Тел.: +79203910501 Реквизиты: ИНН: 7 751 312 214 КПП: 775 101 001 ОГРН: 1 247 700 357 134 ОКПО: 51 541 648
Директор	
_____/О.А.Клубничкина/	Руководитель
«___» _____ 2026 г.	_____/Турмалин Т./
МП	«___» _____ 2026г.
	МП

Рисунок 6 - заполненный документ

3 АНАЛИТИЧЕСКАЯ ЧАСТЬ. САЙТ ДЛЯ ИГРОВОГО СООБЩЕСТВА

3.1 Описание предметной области

Laravel — это фреймворк для веб-приложений с выразительным и элегантным синтаксисом [7]. Веб-фреймворк предоставляет структуру и отправную точку для создания приложения, позволяя сосредоточиться на создании чего-то потрясающего. Laravel предоставляет мощные функции, такие как глубокое внедрение зависимостей, выразительный уровень абстракции базы данных, очереди и запланированные задания, модульное и интеграционное тестирование и многое другое. В данной работе ларавел рассматривается в совокупности с инструментами Breeze + inertia и vue. Inertia.js выступает в роли связующего звена, позволяя создавать современные одностраничные приложения (SPA) с использованием классической серверной маршрутизации и контроллеров Laravel. Интеграция с Vue.js обеспечивает реактивный и компонентный подход к построению пользовательского интерфейса, делая взаимодействие с системой быстрым и интуитивно понятным. Подобный стек технологий значительно упрощает процесс разработки, устраняя необходимость в создании отдельного сложного API и позволяя команде оставаться в привычной экосистеме PHP. В результате совместное использование этих инструментов гарантирует высокую производительность, отличную поддерживаемость кода и превосходный пользовательский опыт конечного продукта.

3.2 Обзор существующих решений

Blade и Vue — это инструменты для разных уровней веб-разработки [8, 9]. Blade работает на сервере (бэкенд), собирая HTML-страницу, а Vue работает в браузере (фронтенд), создавая интерактивный и динамичный интерфейс. Laravel Breeze — это минималистичный и простой стартовый набор (skeleton) от создателей фреймворка Laravel. Он предоставляет базовый функционал

для аутентификации пользователей: вход, регистрацию, сброс пароля, подтверждение почты и управление профилем.

Характеристика	Blade	Vue
Где выполняется	Сервер (PHP)	Создание интерактивных компонентов и SPA
Перезагрузка страницы	При каждом переходе страница перезагружается полностью	Изменения происходят без перезагрузки (SPA — Single Page Application)
Связан с фреймворком	Является стандартным шаблонизатором фреймворка Laravel	Самостоятельный JavaScript-фреймворк (часто идет в связке с Laravel)
SEO (продвижение)	Идеально для поисковиков, так как код готов сразу на сервере	Требует настройки (SSR), так как изначально рендерится в браузере

Таблица 5 - сравнение blade и vue

3.3 Формирование требований к системе

Для работы с Laravel [10] требуется установить php, composer и npm:

1. `/bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.5)"`
2. `composer global require laravel/installer`
3. `laravel new example-app`

4. `cd example-app`
5. `npm install && npm run build`
6. `composer run dev`

В проектах на Laravel npm используется для управления JavaScript-пакетами и компиляции фронтенд-ресурсов (CSS и JS). Composer — это менеджер зависимостей для PHP. В Laravel он необходим для автоматической установки фреймворка, загрузки сторонних библиотек, управления их версиями, а также для автоматической загрузки (autoloading) всех классов в проекте.

Для разработки спользуем **Laravel Breeze (Inertia + Vue)** [11] — это стандарт.

1. `composer require inertiajs/inertia-laravel` - установка inertia.
2. `npm install -g @vue/cli` — установка vue.
3. `npm install @inertiajs/vue3 @inertiajs/vite` — установка inertia.

4 ПРОЕКТНАЯ ЧАСТЬ. Реализация индивидуальной темы на Laravel

4.1 Проектирование архитектуры приложения (компонентная диаграмма)

На рисунке 3 изображена структура сайта по генерации персонажа для настольной ролевой игры D&D. Он должен включать в себя: регистрацию и авторизацию, страницы которых breeze уже включает в себя, сам скрипт для генерации и визуализация сгенерированного контента. У пользователя есть возможность вписать имя персонажа, после чего генерируется карточка персонажа. Нейронные сети использоваться для проекта не будут, соответственно, для создания функции генерации на сайте нужно будет прописать скрипт, который будет брать данные, заранее записанные в базу данных, и показывать их в поле таблицы персонажа на самом сайте. Редактирование контента пользователем не подразумевается. Для генерации персонажа от пользователя требуется только вписать имя будущего персонажа. Все персонажи и пользователи сохраняются в базе данных, а также видны в админпанели.

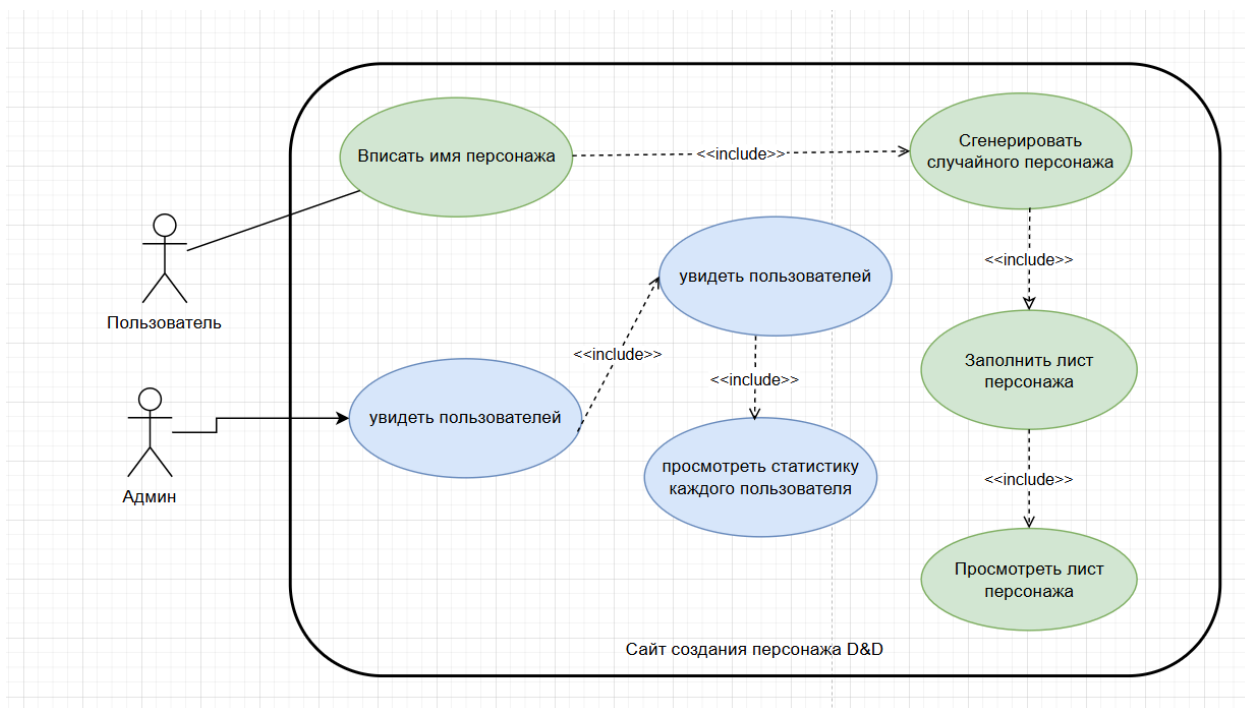


Рисунок 7 - функционал сайта

4.2 Проектирование базы данных (ER-диаграмма)

База данных выполняет функцию хранения всех пользователей, их персонажей и данные для создания персонажей .

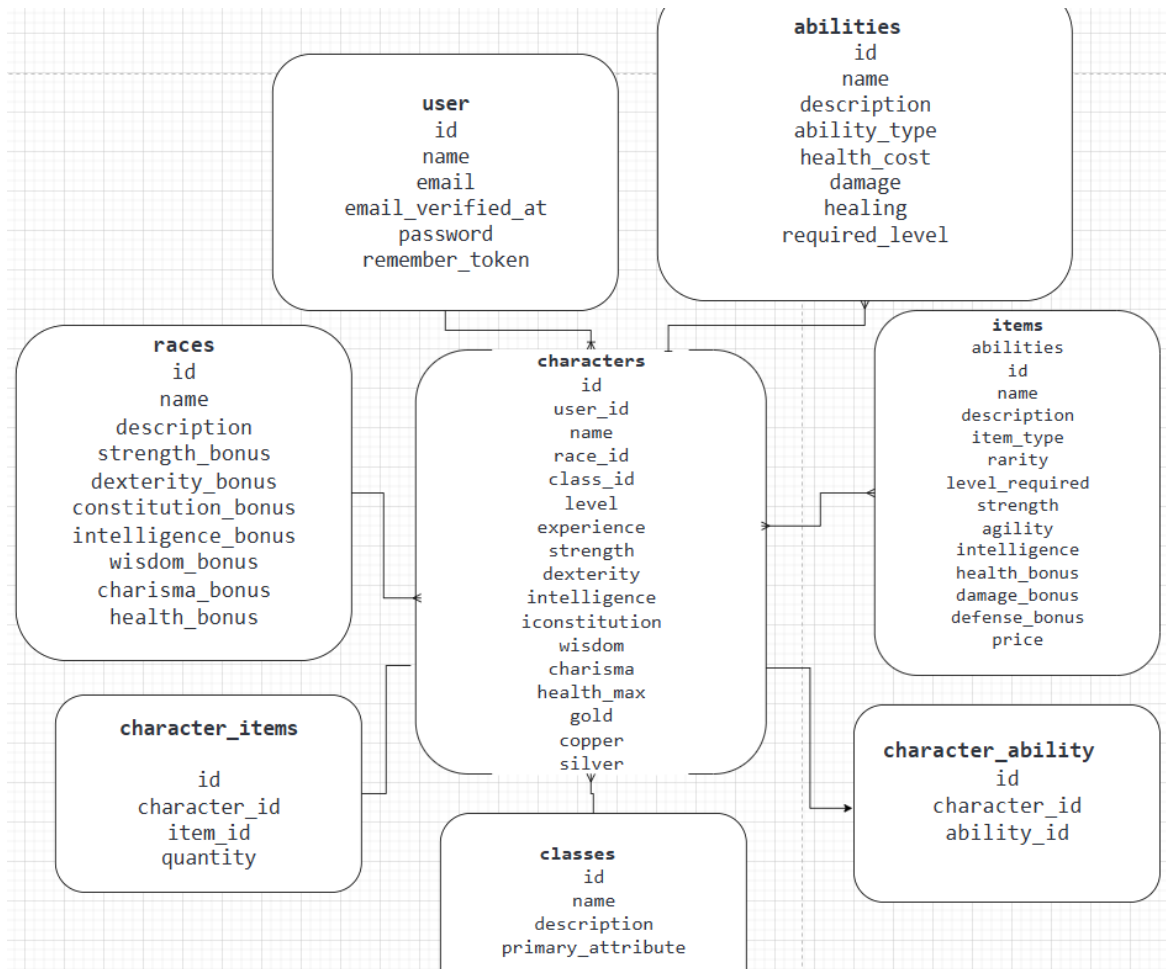


Рисунок 8 - ER диаграмма для сайта-генератора персонажей

Админ-панель							Выйти из таверны
ADMIN ADMIN@gmail.com ID: 3 Персонажей: 1							
Имя	Раса	Класс	Уровень	Сила	Ловк.	Здоровье	Золото
DAS	табакси	Бард	1	17	8	8	13

NYV NYV@gmail.com ID: 1 Администратор Персонажей: 1							
Имя	Раса	Класс	Уровень	Сила	Ловк.	Здоровье	Золото
CHAMBER	Эльф	Бард	1	16	12	14	14

Рисунок 9 - отображение базы данных на сайте

4.3 Реализация функциональных модулей (листинги)

Маршрутизация (Routing). Файл `routes/web.php` определяет соответствие между URL-адресами и действиями контроллеров. Laravel поддерживает REST-ресурсную маршрутизацию, группы с `middleware`, привязку моделей (Route Model Binding), что сокращает объем шаблонного кода. Для создания роута, в первую очередь требуется создать контроллер, после чего подключить его: `use App\Http\Controllers\Auth\AuthenticatedSessionController;`

Контроллеры (Controllers). Служат связующим звеном между моделями и представлениями. Laravel предлагает генерацию контроллеров через Artisan-команды, автоматическое внедрение зависимостей (Dependency Injection) и использование инъекции запросов (Form Requests) для валидации.

Листинг 4 - контроллер входа в аккаунт

<https://github.com/metyloran/kursovaya/blob/main/list4>

Модели и ORM Eloquent. Eloquent реализует паттерн ActiveRecord, где каждая модель соответствует таблице в БД. Разработчик работает с объектами и их методами (`$user->posts()->where('active', 1)->get()`) вместо написания сырых SQL-запросов. ORM также поддерживает отношения (`belongsTo`, `hasMany`, `belongsToMany`), события моделей и доступ к коллекциям.

Листинг 5 - модель для таблицы `character`

<https://github.com/metyloran/kursovaya/blob/main/list5>

Миграции и сиды (Migrations & Seeders). Система миграций позволяет версионировать схему базы данных, а сиды — наполнять её тестовыми данными. Это критически важно для командной разработки и CI/CD.

Листинг 6 - миграция для создания таблицы персонажа

<https://github.com/metyloran/kursovaya/blob/main/list6>

Middleware. Слой, который обрабатывает HTTP-запрос до того, как он достигнет контроллера. Типовые `middleware`: аутентификация, CORS, защита от CSRF, сжатие ответов.

Листинг 7 - обработчик запросов на вход в админ панель

<https://github.com/metyloran/kursovaya/blob/main/list7>

Сервис-провайдеры (Service Providers). Центральный механизм конфигурации и загрузки компонентов приложения. Почти все встроенные сервисы (базы данных, очереди, почта, кэш) регистрируются через провайдеров [12].

Листинг 8 - главный сервис сайта

<https://github.com/metyloran/kursovaya/blob/main/list8>

4.4 Реализация процедур и триггеров

Триггеры и процедуры прописываются непосредственно в базе данных. Представленный ниже триггер выполняется автономно, каждый раз как пользователь добавляет и превышает определенное количество опыта, уровень персонажа повышается и изменения сохраняются в таблице логов.

Листинг 9 - триггер обновления столбца

<https://github.com/metyloran/kursovaya/blob/main/list9>

Процедура, ниже приведенная, выполняется каждый раз, когда пользователь заходит на сайт и создает персонажа. В таблице users в колонке last_activity отображается последняя активность и именно она изменяется посредством функционирования процедуры.

Листинг 10 - процедура

<https://github.com/metyloran/kursovaya/blob/main/list10>

4.5 Обеспечение безопасности и авторизации

Обеспечение безопасности и авторизация заранее встроены в breeze, все скачанные компоненты остается только подвести под стиль сайта и вписать в роуты.

Листинг 11 - путь подключения авторизации

<https://github.com/metyloran/kursovaya/blob/main/list11>

контроллер авторизации также появляются при скачивании breeze, остается только немного изменить.

Листинг 12 - контроллер авторизации

<https://github.com/metyloran/kursovaya/blob/main/list12>

The image displays two side-by-side web forms. The left form is titled 'Добро пожаловать' (Welcome) and features a crossed-swords icon. It includes a login section with a text input for 'Имя искателя приключений' (Adventurer's name) containing 'Эльдрин, Брунхильда...' and a password input labeled 'Тайное слово' (Secret word) with placeholder text 'ваш пароль...'. A 'Войти в мир приключений' (Log in to the world of adventures) button is at the bottom. The right form is titled 'Регистрация' (Registration) with the subtitle 'Начни своё приключение в мире D&D'. It contains fields for 'Имя искателя' (Adventurer's name) with 'Арагорн, Морриган...', 'Голубиная почта' (Email) with 'hero@dungeon.com', 'Тайное слово' (Secret word) with placeholder 'придумайте пароль...', and a confirmation field 'Подтверждение тайного слова' (Confirm secret word) with placeholder 'подтвердите пароль...'. A 'Создать героя' (Create hero) button is at the bottom. Both forms have links at the bottom: 'Ещё не зарегистрированы? Стол регистрации' (Not registered? Registration desk) on the left and 'Уже покоряете подземелья? Войти в таверну' (Already conquering dungeons? Log in to the tavern) on the right.

Рисунок 10 - форма регистрации и авторизации

4.6 Тестирование и отладка

Laravel предоставляет готовую инфраструктуру для автоматизированного тестирования на основе фреймворка PHPUnit. После создания проекта автоматически формируется каталог tests, который содержит две основные группы тестов: Unit и Feature. Unit-тесты предназначены для проверки отдельных методов и компонентов приложения в изоляции, тогда как Feature-тесты позволяют тестировать взаимодействие различных модулей системы, маршрутов, контроллеров и базы данных. Для проведения тестирования была создана отдельная среда выполнения с использованием файла .env.testing [13], что позволило исключить влияние тестов на рабочую базу данных проекта. Во время выполнения тестов Laravel автоматически

использует окружение `testing`, а также временные настройки кэша и сессий. Для выполнения тестов использовались встроенные средства Laravel, позволяющие имитировать HTTP-запросы и анализировать ответы сервера. Особое внимание уделялось проверке операций с базой данных. В процессе тестирования контролировалась корректность:

1. создания записей;
2. обновления информации;
3. удаления данных;
4. выполнения миграций;
5. работы связей между моделями.

Использование тестовой базы данных позволило выполнять проверки без риска повреждения основной информации проекта.

В результате пользователь получает карточку персонажа с данными для последующей игры. Создание изображения на сайте не предусматривается, так как карточка создается для настольно ролевой игры, где человек должен самостоятельно вжиться в роль и придумать образ.



Рисунок 11 - карточка персонажа

ЗАКЛЮЧЕНИЕ

В первой части работы был разработан веб-сайт документооборота на основе фреймворка Django. Основной функционал системы заключается в автоматическом формировании документов на основании данных, введенных пользователем через веб-интерфейс. После заполнения формы система обрабатывает полученные данные, подставляет их в шаблон документа и предоставляет готовый файл для скачивания.

Использование Django позволило значительно ускорить процесс разработки благодаря встроенным средствам администрирования, системе ORM и готовым механизмам обеспечения безопасности. В результате была создана функциональная и масштабируемая основа веб-приложения, отвечающая поставленным требованиям.

Во второй части работы был разработан веб-сайт на базе фреймворка Laravel, предназначенный для генерации персонажей. Пользователь вводит имя персонажа, после чего система автоматически формирует его описание на основе заданных алгоритмов генерации. Были изучены особенности архитектуры Laravel, принципы организации MVC, работа с маршрутами, контроллерами, моделями и системой шаблонов Breeze, vue, inertia.

В процессе разработки были реализованы основные функции приложения, настроено взаимодействие с базой данных, выполнены тестирование и отладка программного продукта. Использование Laravel обеспечило удобную организацию кода, высокую производительность разработки и возможность дальнейшего расширения функционала системы. Проведенное тестирование подтвердило корректность работы реализованных модулей и устойчивость приложения к типовым пользовательским действиям.

Результат работы

В результате выполнения курсовой работы были изучены современные технологии разработки веб-приложений на базе фреймворков Django и Laravel. Выполнено проектирование структуры системы, реализованы

основные функциональные возможности приложения, настроено взаимодействие с базой данных и проведено комплексное тестирование.

Практическая реализация проекта позволила закрепить навыки разработки серверной части веб-приложений, работы с архитектурой MVC, проектирования баз данных, организации маршрутизации запросов, а также тестирования и отладки программного обеспечения. Все поставленные в рамках курсовой работы задачи были успешно выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методы HTTP запросов. – URL:
<https://developer.mozilla.org/ru/docs/Web/HTTP/Reference/Methods> (дата обращения: 05.06.2026).
2. Установка и использование менеджера пакетов pip. – URL:
<https://docs-python.ru/tutorial/instrumenty-razrabotki-python/ustanovka-ispolzovanie-menedzhera-paketov-pip/> (дата обращения: 05.06.2026).
3. Понин Ф.Н. Методология проектирования и создания баз данных для современного программного обеспечения // Universum: технические науки. – 2024. – №1(118). – URL:
<https://cyberleninka.ru/article/n/metodologiya-proektirovaniya-i-sozdaniya-baz-dannyh-dlya-sovremennogo-programmnogo-obespecheniya> (дата обращения: 05.06.2026).
4. Модели Django. – URL:
https://developer.mozilla.org/ru/docs/Learn_web_development/Extensions/Server-side/Django/Models (дата обращения: 05.06.2026).
5. Руководство по Django. Часть 4. Формы и обобщенные представления. – URL: <https://docs.djangoproject.com/en/6.0/intro/tutorial04/> (дата обращения: 05.06.2026).
6. Lev_Antropov. Архитектура универсального проекта Django // Хабр. – 2022. – URL: <https://habr.com/ru/articles/692518/> (дата обращения: 05.06.2026).
7. Установка. – URL: <https://laravel.ru/docs/12.x/installation> (дата обращения: 05.06.2026).
8. Документация о Vue. – URL: <https://vuejs.org/guide/introduction.html> (дата обращения: 05.06.2026).
9. Шаблоны Blade. – URL: <https://laravel.ru/docs/12.x/blade> (дата обращения: 05.06.2026).

10. Документация Laravel. – URL: <https://laravel.com/docs/13.x> (дата обращения: 05.06.2026).
11. М. R. Установка готового решения для аутентификации и авторизации на php: Laravel Breeze, Inertia.js, Vue 3 // Kaktotak.by. – 2024. – URL: <https://kaktotak.by/internet-biznes/ustanovka-platformy-s-autentifikatsiei-gotovloe-reshenie-dlia-bystrogo-starta-svoei-crm-sistemy-laravel-breeze-inertia-js-vue-3-i-tailwind-css> (дата обращения: 05.06.2026).
12. Goodness Woke. Лучшие практики Laravel для разработчиков в 2024 году // Хабр. – 2024. – URL: <https://habr.com/ru/articles/814843/> (дата обращения: 05.06.2026).
13. Тестирование моделей в Laravel. – URL: <https://laravel.demiart.ru/testing-models/> (дата обращения: 05.06.2026).
14. Исходный код сайта на Django. – URL: <https://drive.google.com/drive/folders/1OObUFRFawvPUPo5O6GwMfxt3eJeDJPg0?usp=sharing> (дата обращения: 05.06.2026).
15. Исходный код сайта на Laravel. – URL: <https://drive.google.com/drive/folders/1bFjVb4iQutQSr31LOd3pXW1lpu0uezN?usp=sharing> (дата обращения: 05.06.2026).